1. Understand Hashing, Bucket Concept, and Collision (Beginner Level)

What is Hashing?

Hashing is a technique used by HashMap and HashSet to store and retrieve data quickly.

It converts a key into an integer called a **hash code** using the hashCode() method.

```
String key = "Java";

int hash = key.hashCode();
```

The hash code determines **where the key-value pair should be stored**.

What is a Bucket?

A **bucket** is a storage location inside a HashMap.

The generated hash code is converted into a bucket index.

```
Key

↓

hashCode()

↓

Bucket Number

↓

Store Entry
```

Example:

```
Bucket 0 → Empty
Bucket 1 → (101, Ali)
Bucket 2 → Empty
Bucket 3 → (102, Ahmed)
```

What is a Collision?

A **collision** occurs when **two different keys are assigned to the same bucket**.

```
"ABC"

↓

Bucket 2

"XYZ"

↓

Bucket 2
```

Both keys share the same bucket.

HashMap handles this internally and still retrieves the correct value.

Important Interview Points ⭐

- **Hashing** converts a key into a hash code for fast storage and retrieval.
- A **bucket** is the location where entries are stored inside a HashMap.
- Different keys can sometimes produce the same bucket (**collision**).
- HashMap resolves collisions using equals() after locating the bucket.
- Good hashing distributes keys evenly across buckets, improving performance.
- Hash collisions are **normal** and expected—they do not mean the map is broken.

Tricky Interview Questions

Q1. What is hashing?

Answer:

A technique that converts a key into a hash code for efficient storage and lookup.

Q2. What is a bucket?

Answer:

A storage location inside a HashMap where key-value pairs are stored.

Q3. What is a collision?

Answer:

When two different keys are placed in the same bucket.

Q4. Are collisions errors?

Answer: No.

They are expected and handled internally by HashMap.

Q5. Which methods help resolve collisions?

Answer:

- hashCode()
- equals()

Interview Summary ⭐

- **Hashing** converts keys into hash codes.
- **Buckets** store key-value pairs inside a HashMap.
- **Collisions** occur when different keys share the same bucket.
- HashMap uses hashCode() to locate a bucket and equals() to identify the correct key.
- Efficient hashing enables **O(1) average** lookup performance.

> **Interview One-Liner:**
>
> **"HashMap uses hashing to place entries into buckets, and if multiple keys land in the same bucket, it resolves the collision using equals()."**

2. Understand Why Immutable Keys Are Safer in HashMap

What is an Immutable Key?

An **immutable key** is a key whose value **cannot change after being created**.

Example:

```
String key = "Java";
```

String is immutable.

Why Are Immutable Keys Safer?

When a key is inserted into a HashMap, its hashCode() determines its bucket.

If the key changes later, its hash code may also change.

The object remains in the old bucket, but future searches use the new hash code, so the value cannot be found.

Example

```
HashMap map = new HashMap<>();

map.put("Java", 100);
```

Searching:

```
map.get("Java");
```

Works correctly because "Java" never changes.

Important Interview Points ⭐

- A HashMap relies on a key's **hashCode()** remaining unchanged.
- Immutable objects produce a **consistent hash code** throughout their lifetime.
- String is the most commonly used HashMap key because it is immutable.
- Mutable keys can make stored values difficult or impossible to retrieve.
- Wrapper classes like Integer and Long are also immutable and safe as keys.

Tricky Interview Questions

Q1. Why is String commonly used as a HashMap key?

Answer:

Because it is immutable, ensuring a stable hash code.

Q2. Can mutable objects be used as HashMap keys?

Answer: Yes, but it is not recommended.

Changing the key after insertion can break lookups.

Q3. What happens if a key changes after insertion?

Answer:

The object remains in its original bucket, but future searches may fail because the hash code has changed.

Q4. Which types are considered safe HashMap keys?

Answer:

Immutable types like:

- String
- Integer
- Long
- UUID

Interview Summary ⭐

- **Immutable keys provide stable hash codes**, making HashMap lookups reliable.
- Mutable keys can cause lookup failures if modified after insertion.
- String is the preferred key type because it is immutable.
- Choosing immutable keys ensures consistent and predictable HashMap behavior.

> **Interview One-Liner:**
>
> **"Immutable keys are safer because their hash code never changes, allowing HashMap to locate stored values reliably."**

---

3. Use containsKey(), getOrDefault(), putIfAbsent(), and remove()

Commonly Used HashMap Methods

```
HashMap map = new HashMap<>();
```

---

1. containsKey()

Checks whether a key exists.

```
map.containsKey(101);
```

Returns:

```
true / false
```

---

2. getOrDefault()

Returns the value if the key exists; otherwise returns a default value.

```
map.getOrDefault(101, "Not Found");
```

---

3. putIfAbsent()

Adds a key-value pair **only if the key is not already present**.

```
map.putIfAbsent(101, "Ali");
```

---

4. remove()

Removes a key-value pair.

```
map.remove(101);
```

Important Interview Points ⭐

- containsKey() checks key existence without retrieving the value.
- getOrDefault() avoids null by returning a fallback value.
- putIfAbsent() prevents overwriting an existing value.
- remove() deletes a mapping by key.
- These methods make HashMap code safer, cleaner, and easier to read.

Tricky Interview Questions

Q1. Difference between get() and containsKey()?

Answer:

- get() returns the value (or null).
- containsKey() only checks whether the key exists.

Q2. Why use getOrDefault() instead of get()?

Answer:

To avoid handling null manually when a key is missing.

Q3. Does putIfAbsent() replace an existing value?

Answer: No.

It inserts only if the key is absent.

Q4. Does remove() return anything?

Answer: Yes.

It returns the removed value or null if the key doesn't exist.

Interview Summary ⭐

- containsKey() checks for key existence.
- getOrDefault() returns a default value if the key is missing.
- putIfAbsent() inserts only when a key doesn't already exist.
- remove() deletes a key-value mapping.
- These methods help write **safe and concise** HashMap code.

> **Interview One-Liner:**
>
> **"containsKey(), getOrDefault(), putIfAbsent(), and remove() are essential HashMap methods for safely checking, retrieving, inserting, and deleting key-value pairs."**

---

4. Understand HashMap Does Not Guarantee Iteration Order

Does HashMap Maintain Order?

No.

HashMap **does not guarantee any iteration order**.

```
HashMap map = new HashMap<>();

map.put(3, "C");
map.put(1, "A");
map.put(2, "B");

System.out.println(map);
```

Output may be:

```
{2=B, 3=C, 1=A}
```

or another order.

Why?

HashMap stores entries based on their **hash values**, not the order they were inserted.

As a result, iteration order is unpredictable and may change.

If Order Is Needed

- **Insertion Order** → LinkedHashMap
- **Sorted Order** → TreeMap

Important Interview Points ⭐

- HashMap **does not preserve insertion order**.
- Iteration order depends on the internal hash table structure.
- The order should **never be relied upon** in application logic.
- Use LinkedHashMap if insertion order matters.
- Use TreeMap if sorted key order is required.

Tricky Interview Questions

Q1. Does HashMap maintain insertion order?

Answer: No.

Q2. Which Map preserves insertion order?

Answer: LinkedHashMap.

Q3. Which Map stores keys in sorted order?

Answer: TreeMap.

Q4. Can HashMap iteration order change?

Answer: Yes.

The order may vary between executions or after modifications.

Q5. Is HashMap unordered or randomly ordered?

Answer:

Neither exactly. It is **unordered**—its iteration order is determined by the internal hashing structure and should not be considered random or predictable.

Interview Summary ⭐

- **HashMap does not guarantee iteration or insertion order.**
- Entries are organized according to their hash values.
- Never write code that depends on HashMap iteration order.
- Use **LinkedHashMap** for insertion order and **TreeMap** for sorted order.
- Choosing the right Map implementation depends on whether ordering is required.

> **Interview One-Liner:**
>
> **"HashMap is optimized for fast key-based lookup, not ordering, so its iteration order is undefined and should never be relied upon."**